

DarshanEase – Temple Darshan Ticket Booking System

Final Year CSE Project Documentation

Prepared For:

Department of Computer Science Engineering

Project Type:

Full Stack Web Application

Technology Stack:

MERN Stack (MongoDB, Express.js, React.js, Node.js)

Submitted by:

Name : Pavada Samuel Abhishek,

Roll no : 323506402384

Table of Contents

1. Introduction
 2. Overall Description
 3. System Architecture
 4. Functional Requirements
 5. Non-Functional Requirements
 6. Database Design
 7. Use Case Diagram Description
 8. Data Flow Diagram (DFD)
 9. API Requirements
 10. User Interface Requirements
 11. Security Requirements
 12. Future Enhancements
 13. Conclusion
-

1. Introduction

1.1 Purpose of the Document

This Software Requirements Specification (SRS) document provides a detailed description of the requirements, functionality, architecture, and design considerations of **DarshanEase – Temple Darshan Ticket Booking System**.

The purpose of this document is to define the expected behavior of the application and provide a common understanding between developers, testers, project guides, and stakeholders.

This document follows IEEE recommended SRS guidelines and acts as a reference throughout the software development lifecycle.

1.2 Scope of the Project

DarshanEase is a full-stack web application designed to simplify the process of temple visit planning and darshan ticket booking.

Traditional temple ticket booking systems often require devotees to physically visit counters or use multiple platforms to obtain information. DarshanEase provides a centralized digital platform where users can:

- Explore different temples
- View temple information
- Check available darshan slots
- Book darshan tickets online
- Manage booking history
- Receive booking confirmations

The system also provides an administrative interface where authorized administrators can manage temples, darshan schedules, users, and bookings.

1.3 Project Objectives

The major objectives of DarshanEase are:

Objective	Description
Digital Booking	Provide online darshan ticket booking facility
Accessibility	Allow devotees to access temple services anytime
Slot Management	Provide organized darshan schedule management

Objective	Description
Automation	Reduce manual booking processes
Data Management	Maintain secure records of users and bookings
User Experience	Provide simple and responsive interface

1.4 Intended Audience

This document is intended for:

- Software developers
 - Project guides
 - Testing teams
 - Database designers
 - Future maintenance teams
 - Academic reviewers
-

1.5 Definitions and Abbreviations

Term	Meaning
------	---------

SRS	Software Requirements Specification
MERN	MongoDB, Express.js, React.js, Node.js
API	Application Programming Interface
JWT	JSON Web Token
UI	User Interface
DFD	Data Flow Diagram
CRUD	Create, Read, Update, Delete
DB	Database

2. Overall Description

2.1 Product Perspective

DarshanEase follows a three-layer web application architecture:

1. Presentation Layer

2. Application Layer

3. Data Layer

The system uses:

- React.js for frontend development
 - Express.js and Node.js for backend services
 - MongoDB for database storage
 - JWT for authentication
-

2.2 Product Features

The major features of the system are:

User Features

- User registration
- Secure login
- Temple browsing
- Temple searching
- Temple details viewing
- Darshan slot checking
- Ticket booking
- Booking history
- Booking cancellation
- Notification confirmation

Admin Features

- Secure admin login
 - Add temples
 - Update temples
 - Delete temples
 - Manage darshan slots
 - View bookings
 - Update booking status
 - Manage users
-

2.3 User Classes and Characteristics

User / Devotee

Characteristics:

- General visitors
- Requires simple navigation
- Interested in temple information and booking services

Administrator

Characteristics:

- Authorized system manager
 - Responsible for maintaining application data
 - Requires additional privileges
-

2.4 Operating Environment

Hardware Requirements

Component Requirement

Processor Intel i3 or above

RAM Minimum 4GB

Storage Minimum 10GB

Software Requirements

Component Technology

Frontend React.js + Vite

Backend Node.js + Express.js

Database MongoDB

Browser Chrome/Firefox/Edge

OS Windows/Linux/macOS

2.5 Design and Implementation Constraints

- Requires internet connection
 - MongoDB availability is required
 - JWT security depends on token management
 - Admin operations require authentication
 - System performance depends on server resources
-

2.6 Assumptions and Dependencies

Assumptions:

- Users provide correct information
- Administrators maintain valid temple details
- Internet services remain available

Dependencies:

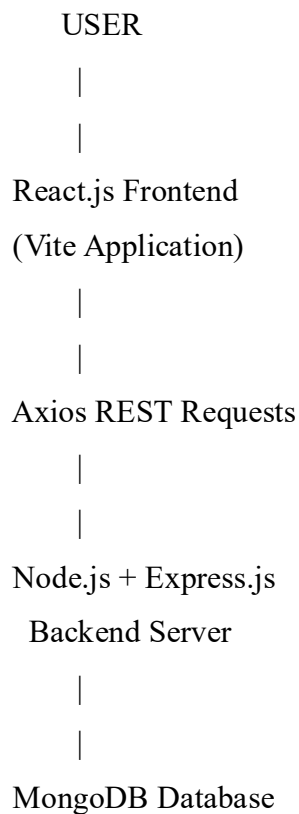
- MongoDB database service
 - Node.js runtime environment
 - Third-party notification/payment services
-

3. System Architecture

3.1 MERN Stack Architecture

DarshanEase follows MERN architecture.

Architecture Diagram



3.2 Frontend Layer

Technology:

- React.js
- Bootstrap / Material UI
- Axios
- React Router DOM
- React Toastify

Responsibilities:

- Display user interface
 - Handle navigation
 - Validate forms
 - Communicate with backend APIs
-

3.3 Backend Layer

Technology:

- Node.js
- Express.js

Responsibilities:

- Handle API requests
 - Authentication
 - Authorization
 - Business logic
 - Database communication
-

3.4 Database Layer

Technology:

MongoDB

Responsibilities:

- Store user information
 - Store temple details
 - Store bookings
 - Maintain darshan schedules
-

4. Functional Requirements

4.1 User Module

FR-01 User Registration

The system shall allow new users to create an account.

Input:

- Name
- Email
- Password

Output:

- Account creation confirmation
-

FR-02 User Login

The system shall authenticate registered users.

Process:

1. User enters credentials
 2. Backend verifies information
 3. JWT token is generated
 4. User accesses protected features
-

FR-03 Browse Temples

Users shall be able to view available temples.

Information displayed:

- Temple name
 - Location
 - Image
 - Description
-

FR-04 Search and Filter Temples

Users can search temples based on:

- Name
 - Location
 - Availability
-

FR-05 View Temple Details

Users can view:

- Temple history
 - Timings
 - Location
 - Darshan information
-

FR-06 View Darshan Slots

Users can check:

- Available dates
 - Available timings
 - Remaining seats
-

FR-07 Book Darshan Tickets

Users can:

- Select temple
 - Select slot
 - Confirm booking
-

FR-08 Booking History

Users can view:

- Previous bookings
 - Current bookings
 - Booking status
-

FR-09 Cancel Booking

Users can cancel eligible bookings.

4.2 Admin Module

FR-10 Admin Login

Admin can securely access dashboard.

FR-11 Temple Management

Admin can:

- Add temple
 - Edit temple
 - Delete temple
-

FR-12 Darshan Slot Management

Admin can:

- Create slots
 - Modify availability
 - Remove slots
-

FR-13 Booking Management

Admin can:

- View all bookings
 - Approve bookings
 - Reject bookings
 - Update status
-

FR-14 User Management

Admin can:

- View users
 - Manage accounts
-

5. Non-Functional Requirements

5.1 Performance Requirements

- Application should load quickly
 - APIs should respond efficiently
 - Database queries should be optimized
-

5.2 Security Requirements

- Password encryption
- JWT authentication

- Role-based authorization
 - Secure API endpoints
-

5.3 Usability Requirements

- Simple interface
 - Responsive design
 - Mobile-friendly pages
-

5.4 Scalability

The application should support:

- Increasing users
 - Additional templates
 - More booking records
-

5.5 Reliability

The system should:

- Prevent duplicate bookings
 - Maintain accurate records
-

5.6 Maintainability

The application should follow:

- Modular coding structure
 - Reusable components
 - Proper documentation
-

5.7 Availability

The system should provide continuous access with minimum downtime.

6. Database Design

MongoDB uses collections to store application data.

Users Collection

Field	Description
userId	Unique user identifier
name	User name
email	User email
password	Encrypted password
role	User/Admin

Temples Collection

Field	Description
templeId	Unique temple ID
templeName	Temple name
location	Temple location
description	Temple details
image	Temple image
timings	Opening timings

DarshanSlots Collection

Field	Description
slotId	Slot identifier
templeId	Related temple
date	Darshan date
time	Darshan timing
availability	Available seats

Bookings Collection

Field	Description
bookingId	Booking identifier
userId	User reference

Field	Description
templeId	Temple reference
slotId	Slot reference
bookingDate	Booking date
status	Booking status

Donations Collection

Field	Description
donationId	Donation identifier
userId	Donor reference
amount	Donation amount
date	Donation date

7. Use Case Diagram Description

User Use Cases

Actor:

User

Actions:

User

|

|-- Register

|

|-- Login

|

|-- Browse Temples

|

|-- View Details

|

|-- Search Temple

|

|-- View Slots

|
|-- Book Ticket
|
|-- View Booking History
|
|-- Cancel Booking

Admin Use Cases

Actor:

Admin

Actions:

Admin

|
|-- Login
|
|-- Add Temple
|
|-- Update Temple
|
|-- Delete Temple
|
|-- Manage Slots
|
|-- View Bookings
|
|-- Manage Users

8. Data Flow Diagram

Level 0 DFD

User

|
|

DarshanEase System

|
|

Database

Level 1 DFD

User

|
|

Authentication Process

|

Users Database

User

|
|

Temple Search Process

|

Temple Database

User

|
|

Booking Process

|

Booking Database

Admin

|
|

Management Process

|

9. API Requirements

Authentication APIs

Register API

Method:

POST

Endpoint:

/api/auth/register

Purpose:

Create user account.

Login API

Method:

POST

Endpoint:

/api/auth/login

Purpose:

Authenticate user.

Temple APIs

Get Temples

GET

/api/temples

Add Temple

POST

/api/temples

Update Temple

PUT

/api/temples/:id

Delete Temple

DELETE

/api/temples/:id

Booking APIs

Create Booking

POST

/api/bookings

View Bookings

GET

/api/bookings

Update Booking Status

PUT

/api/bookings/:id

10. User Interface Requirements

Home Page

Contains:

- Navigation bar
 - Temple highlights
 - Introduction
 - Booking options
-

Temple Listing Page

Contains:

- Temple cards
 - Search option
 - Filters
-

Temple Details Page

Contains:

- Temple information
 - Images
 - Timings
 - Available slots
-

Booking Page

Contains:

- Slot selection
 - User details
 - Booking confirmation
-

My Bookings Page

Displays:

- Booking history
 - Status
 - Cancellation option
-

Admin Dashboard

Contains:

- Temple management
 - Booking management
 - User management
-

11. Security Requirements

Password Encryption

Passwords must be stored securely using encryption algorithms.

JWT Authentication

JWT tokens are used for:

- User verification
 - Secure sessions
 - Protected routes
-

Authorization

Only administrators can access management functions.

Input Validation

System validates:

- User inputs
 - API requests
 - Booking information
-

Secure API Communication

HTTPS communication should be implemented for production deployment.

12. Future Enhancements

Future improvements include:

Online Payment Integration

Integration with:

- Razorpay
- UPI payment gateways

QR Ticket Generation

Generate QR-based entry passes.

Mobile Application

Develop Android/iOS application.

AI Temple Recommendation

Suggest temples based on:

- User interests
- Location
- Previous visits

Real-Time Slot Availability

Provide live slot updates.

13. Conclusion

DarshanEase – Temple Darshan Ticket Booking System provides an efficient digital solution for managing temple visits and darshan bookings.

The MERN stack architecture enables scalability, security, and maintainability. The system reduces manual booking efforts and provides devotees with a convenient platform for planning temple visits.

The application successfully demonstrates the implementation of modern web technologies, database management, authentication mechanisms, and full-stack development practices suitable for a final-year Computer Science Engineering project.
